Initial Database Design

University Library Management System

Course	[Course Code] - Database Systems
Student Name	[Student Name]
Student ID	STUDENTID
System Title	University Library Management System
Date	October 27, 2026

1. System Description

The University Library Management System manages the catalog, circulation, and member services of a single-campus university library. Its purpose is to replace paper cards and spreadsheets with a reliable relational database that records which books the library owns, who may borrow them, and what is currently on loan, reserved, or overdue. The target users are **library members** (students, faculty, and registered public patrons), who search the catalog and reserve titles, and **library staff** (librarians and clerks), who register members, catalog books and physical copies, check items out and in, and collect fines. The main functions are catalog management, member management, loan (circulation) processing, reservations, fine calculation and payment, and management reporting. The design contains twelve entities with the relationships described in Section 4.

2. Requirements Specification

2.1 Functional Requirements

ID	Requirement
FR-01	The system shall allow staff to register, update, and deactivate library members and assign each member exactly one membership type.
FR-02	The system shall maintain a catalog of books with their authors, publisher, category, ISBN, edition, language, and publication year.
FR-03	The system shall record every physical copy of a book with a unique barcode, shelf location, condition, and availability status.
FR-04	The system shall allow users to search the catalog by title, author, ISBN, or category.
FR-05	The system shall allow staff to check out an available copy to an eligible member and record the loan date, due date, and issuing staff member.
FR-06	The system shall record the return of a copy and update the copy's availability status.
FR-07	The system shall calculate an overdue fine when a copy is returned after its due date.
FR-08	The system shall record fines for damaged or lost copies and the date each fine is paid.
FR-09	The system shall allow a member to reserve a book title when no copy is currently available.
FR-10	The system shall hold a returned copy for the first pending reservation and mark the reservation as Ready.
FR-11	The system shall block a checkout that violates a business rule (loan limit, expired membership, or unpaid fines).
FR-12	The system shall maintain staff accounts with roles and record which staff member processed each loan.
FR-13	The system shall generate reports on overdue loans, most-borrowed titles, and fines collected per period.

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Catalog searches shall return results within 2 seconds for a catalog of up to 500,000 titles (supported by indexes on title, ISBN, and author name).
NFR-02	Security	Access shall be role-based (member, clerk, librarian, administrator); passwords shall be stored as salted hashes and member personal data shall be visible only to staff.
NFR-03	Concurrency	Two staff members shall not be able to check out the same copy at the same time; checkout shall run as a single ACID transaction.
NFR-04	Scalability	The database shall support at least 50,000 members and 1,000,000 loan records without redesign.
NFR-05	Reliability	Automated daily backups shall be taken and the system shall target 99% availability during opening hours.

2.3 Business Rules

ID	Business rule / constraint
BR-01	Loan limit: a member cannot have more active loans than the maximum of their membership type (Student 5, Faculty 10, Public 3).
BR-02	Loan period: the due date is the loan date plus the loan days of the membership type (Student 14, Faculty 30, Public 7).
BR-03	A physical copy can belong to at most one active loan (a loan with no return date) at any time.
BR-04	A member whose membership has expired or whose status is Suspended cannot borrow or reserve books.
BR-05	A member whose unpaid fines total more than PHP 100 cannot borrow until the fines are paid.
BR-06	The overdue fine equals the membership type's fine-per-day rate multiplied by the number of days late.
BR-07	A reserved book is held for 3 days after the member is notified; after that the reservation expires and passes to the next member.
BR-08	Copies marked Reference Only, Lost, or Withdrawn cannot be loaned.
BR-09	Every book must have a publisher, a category, and at least one author; ISBN and barcode values must be unique.
BR-10	A due date and a return date cannot be earlier than the loan date; a fine amount must be greater than zero.
BR-11	A member cannot reserve a title they currently have on loan or already have a pending reservation for.

2.4 Assumptions and Scope

In scope:

- Catalog (books, authors, publishers, categories) and physical copy inventory.
- Member and staff records, membership types and their borrowing limits.
- Checkout, return, reservation, and fine processing with the associated rules.
- Operational reports derived from the stored data.

Out of scope:

- E-books and other digital resources; inter-library loans; book purchasing and budgeting.
- Online payment gateways (fines are recorded as paid, not processed electronically).
- User interface design, RFID hardware, and email/SMS delivery of notices.
- Multi-branch libraries (a single campus library is assumed).

Assumptions: all amounts are in Philippine pesos (PHP); a member holds one membership type at a time; an ISBN identifies one edition of a title; all checkouts are processed by staff (no self-service); one fine record is created per fine incident, so a loan may have several fines.

3. Entity List with Attributes

Legend: **PK** = primary key, **FK** = foreign key, **UQ** = unique (candidate key), **NN** = NOT NULL, **NULL** = nullable. Data types follow standard SQL / MySQL notation.

3.1 MEMBERSHIP_TYPE

Description: Category of membership that defines borrowing rules (Student, Faculty, Public). **Candidate key(s):** type_name.

Attribute	Data type	Key	Null?	Notes
type_id	INT	PK	NN	Auto-generated
type_name	VARCHAR(30)	UQ	NN	e.g. Student
max_loans	TINYINT		NN	Max active loans
loan_days	TINYINT		NN	Loan period in days
fine_per_day	DECIMAL(6,2)		NN	PHP per overdue day

3.2 MEMBER

Description: A person registered to borrow and reserve library materials. **Candidate key(s):** email.

Attribute	Data type	Key	Null?	Notes
member_id	INT	PK	NN	Auto-generated
type_id	INT	FK	NN	-> MEMBERSHIP_TYPE
first_name	VARCHAR(50)		NN	
last_name	VARCHAR(50)		NN	
email	VARCHAR(100)	UQ	NN	
phone	VARCHAR(20)		NULL	Optional
address	VARCHAR(200)		NULL	Optional
join_date	DATE		NN	
expiry_date	DATE		NN	Membership end
status	VARCHAR(10)		NN	Active / Suspended / Expired

3.3 STAFF

Description: Library employee who processes loans and returns. **Candidate key(s):** email.

Attribute	Data type	Key	Null?	Notes
staff_id	INT	PK	NN	Auto-generated
first_name	VARCHAR(50)		NN	
last_name	VARCHAR(50)		NN	
email	VARCHAR(100)	UQ	NN	
role	VARCHAR(30)		NN	Clerk / Librarian / Admin
hire_date	DATE		NN	

3.4 PUBLISHER

Description: Company that publishes books held by the library. **Candidate key(s):** name.

Attribute	Data type	Key	Null?	Notes
publisher_id	INT	PK	NN	Auto-generated
name	VARCHAR(100)	UQ	NN	
country	VARCHAR(50)		NULL	
website	VARCHAR(150)		NULL	

3.5 AUTHOR

Description: Person who wrote one or more books in the catalog. **Candidate key(s):** none (surrogate key only).

Attribute	Data type	Key	Null?	Notes
author_id	INT	PK	NN	Auto-generated
first_name	VARCHAR(50)		NULL	Null for mononyms
last_name	VARCHAR(50)		NN	
birth_year	SMALLINT		NULL	
nationality	VARCHAR(50)		NULL	

3.6 CATEGORY

Description: Subject classification used to group books (e.g. Computer Science). **Candidate key(s):** name.

Attribute	Data type	Key	Null?	Notes
category_id	INT	PK	NN	Auto-generated
name	VARCHAR(60)	UQ	NN	
description	VARCHAR(255)		NULL	

3.7 BOOK

Description: A bibliographic title/edition in the catalog (not a physical item). **Candidate key(s):** isbn.

Attribute	Data type	Key	Null?	Notes
book_id	INT	PK	NN	Auto-generated
isbn	CHAR(13)	UQ	NN	ISBN-13
title	VARCHAR(200)		NN	
edition	VARCHAR(20)		NULL	
publication_year	SMALLINT		NN	
language	VARCHAR(30)		NN	
publisher_id	INT	FK	NN	-> PUBLISHER
category_id	INT	FK	NN	-> CATEGORY

3.8 BOOK_AUTHOR (associative)

Description: Links books to authors; resolves the many-to-many Written_By relationship. **Candidate key(s):** (book_id, author_id) - the composite primary key.

Attribute	Data type	Key	Null?	Notes
book_id	INT	PK, FK	NN	-> BOOK
author_id	INT	PK, FK	NN	-> AUTHOR
author_order	TINYINT		NN	1 = first author

3.9 BOOK_COPY

Description: A physical copy of a book that can be shelved and lent. **Candidate key(s):** barcode.

Attribute	Data type	Key	Null?	Notes
copy_id	INT	PK	NN	Auto-generated
book_id	INT	FK	NN	-> BOOK
barcode	VARCHAR(20)	UQ	NN	
shelf_location	VARCHAR(20)		NN	e.g. QA76-B3
condition_status	VARCHAR(15)		NN	New / Good / Worn / Damaged
availability	VARCHAR(15)		NN	Available / On Loan / Reference Only / Lost / Withdrawn
acquired_date	DATE		NN	

3.10 LOAN (associative)

Description: Records one member borrowing one copy; resolves the many-to-many Borrows relationship. **Candidate key(s):** loan_id only (a member may borrow the same copy repeatedly).

Attribute	Data type	Key	Null?	Notes
loan_id	INT	PK	NN	Auto-generated
copy_id	INT	FK	NN	-> BOOK_COPY
member_id	INT	FK	NN	-> MEMBER
staff_id	INT	FK	NN	-> STAFF (issuer)
loan_date	DATE		NN	
due_date	DATE		NN	
return_date	DATE		NULL	NULL while still on loan

3.11 RESERVATION (associative)

Description: A member's request for a title; resolves the many-to-many Reserves relationship. **Candidate key(s):** reservation_id only.

Attribute	Data type	Key	Null?	Notes
reservation_id	INT	PK	NN	Auto-generated
book_id	INT	FK	NN	-> BOOK
member_id	INT	FK	NN	-> MEMBER
reservation_date	DATETIME		NN	
expiry_date	DATE		NULL	Set when notified
status	VARCHAR(12)		NN	Pending / Ready / Fulfilled / Cancelled / Expired

3.12 FINE

Description: A monetary penalty charged to a member for a particular loan. **Candidate key(s):** fine_id only.

Attribute	Data type	Key	Null?	Notes
fine_id	INT	PK	NN	Auto-generated
loan_id	INT	FK	NN	-> LOAN
fine_type	VARCHAR(10)		NN	Overdue / Damage / Lost
amount	DECIMAL(8,2)		NN	Must be > 0
issued_date	DATE		NN	
paid_date	DATE		NULL	NULL = unpaid

4. Relationship List

ID	Name	Participating entities	Cardinality	Participation	Description
R1	Has_Type	MEMBERSHIP_TYPE - MEMBER	1 : N	MEMBER total; MEMBERSHIP_TYPE partial	One type applies to many members; every member must have a type.
R2	Publishes	PUBLISHER - BOOK	1 : N	BOOK total; PUBLISHER partial	A publisher may have many books; each book has exactly one publisher.
R3	Classifies	CATEGORY - BOOK	1 : N	BOOK total; CATEGORY partial	Each book is placed in one category; a category may be empty.
R4	Written_By	BOOK - AUTHOR	M : N	BOOK total; AUTHOR total	A book has one or more authors and an author has one or more books. **Resolved by BOOK_AUTHOR.**
R5	Has_Copy	BOOK - BOOK_COPY	1 : N	BOOK_COPY total; BOOK partial	Each copy belongs to one book; a book may temporarily have no copies.
R6	Borrows	MEMBER - BOOK_COPY	M : N	MEMBER partial; BOOK_COPY partial	Over time a member borrows many copies and a copy is borrowed by many members. **Resolved by LOAN** (loan_date, due_date, return_date).
R7	Processes	STAFF - LOAN	1 : N	LOAN total; STAFF partial	Each loan is issued by one staff member, who may issue many loans.
R8	Reserves	MEMBER - BOOK	M : N	MEMBER partial; BOOK partial	A member may reserve many titles and a title may be reserved by many members. **Resolved by RESERVATION.**
R9	Incurs	LOAN - FINE	1 : N	FINE total; LOAN partial	A loan may produce zero or more fines (overdue, damage, lost); each fine belongs to one loan.

The three many-to-many relationships (R4, R6, R8) are each resolved into an associative entity (BOOK_AUTHOR, LOAN, RESERVATION). After resolution every relationship in the relational schema is one-to-many.

5. Relational Schema

Notation: underlined = primary key; *italic* = foreign key. Composite key shown as (a, b).

Table	Columns
MEMBERSHIP_TYPE	type_id, type_name, max_loans, loan_days, fine_per_day
MEMBER	member_id, *type_id*, first_name, last_name, email, phone, address, join_date, expiry_date, status
STAFF	staff_id, first_name, last_name, email, role, hire_date
PUBLISHER	publisher_id, name, country, website
AUTHOR	author_id, first_name, last_name, birth_year, nationality
CATEGORY	category_id, name, description
BOOK	book_id, isbn, title, edition, publication_year, language, *publisher_id*, *category_id*
BOOK_AUTHOR	(*book_id*, *author_id*), author_order
BOOK_COPY	copy_id, *book_id*, barcode, shelf_location, condition_status, availability, acquired_date
LOAN	loan_id, *copy_id*, *member_id*, *staff_id*, loan_date, due_date, return_date
RESERVATION	reservation_id, *book_id*, *member_id*, reservation_date, expiry_date, status
FINE	fine_id, *loan_id*, fine_type, amount, issued_date, paid_date

Foreign Key Constraints

Foreign key	References	ON DELETE	Relationship
MEMBER.type_id	MEMBERSHIP_TYPE(type_id)	RESTRICT	R1
BOOK.publisher_id	PUBLISHER(publisher_id)	RESTRICT	R2
BOOK.category_id	CATEGORY(category_id)	RESTRICT	R3

Foreign key	References	ON DELETE	Relationship
BOOK_AUTHOR.book_id	BOOK(book_id)	CASCADE	R4
BOOK_AUTHOR.author_id	AUTHOR(author_id)	RESTRICT	R4
BOOK_COPY.book_id	BOOK(book_id)	RESTRICT	R5
LOAN.copy_id	BOOK_COPY(copy_id)	RESTRICT	R6
LOAN.member_id	MEMBER(member_id)	RESTRICT	R6
LOAN.staff_id	STAFF(staff_id)	RESTRICT	R7
RESERVATION.member_id	MEMBER(member_id)	RESTRICT	R8
RESERVATION.book_id	BOOK(book_id)	RESTRICT	R8
FINE.loan_id	LOAN(loan_id)	RESTRICT	R9

Key Constraints (UNIQUE / CHECK)

- UNIQUE: MEMBERSHIP_TYPE.type_name, MEMBER.email, STAFF.email, PUBLISHER.name, CATEGORY.name, BOOK.isbn, BOOK_COPY.barcode.
- CHECK: LOAN.due_date >= loan_date; LOAN.return_date IS NULL OR return_date >= loan_date; FINE.amount > 0; MEMBER.expiry_date >= join_date.
- CHECK (domain lists): MEMBER.status, BOOK_COPY.availability, BOOK_COPY.condition_status, RESERVATION.status, FINE.fine_type.
- Partial unique index on LOAN(copy_id) WHERE return_date IS NULL enforces BR-03 (one active loan per copy).

Normalization Justification (1NF - 3NF)

1NF: All attributes are atomic and there are no repeating groups. A book's multiple authors are not stored in a single column; they are moved to BOOK_AUTHOR with one row per book-author pair. Every table has a primary key.

2NF: All tables with single-column keys are automatically free of partial dependencies. The only composite key, BOOK_AUTHOR(book_id, author_id), has one non-key attribute, author_order, which depends on the whole key (the position of that author on that book), so no partial dependency exists.

3NF: No non-key attribute depends on another non-key attribute. Examples: publisher name and country are in PUBLISHER, not BOOK (otherwise book_id -> publisher_id -> publisher name would be a transitive dependency); category details are in CATEGORY; max_loans, loan_days, and fine_per_day depend on the membership type, so they are stored in MEMBERSHIP_TYPE instead of MEMBER (otherwise member_id -> type -> limits would be transitive). Loan, member, copy, and staff details are referenced by foreign keys rather than copied into LOAN. Fine payment status is not stored separately because it is determined by paid_date. Every determinant in every table is a candidate key, so the schema is also in BCNF. BOOK_COPY.availability is kept as an operational status flag for fast lookup and is updated in the same transaction as the loan record (NFR-03).

6. Sample Data (optional)

MEMBER

member_id	type_id	first_name	last_name	email	join_date	expiry_date	status
1	1	Maria	Santos	maria.santos@univ.edu	2026-06-01	2027-05-31	Active
2	2	Jose	Reyes	jose.reyes@univ.edu	2024-08-15	2027-08-14	Active
3	3	Ana	Cruz	ana.cruz@mail.com	2025-01-10	2026-01-09	Expired

BOOK

book_id	isbn	title	edition	year	language	publisher_id	category_id
1	9780078022159	Database System Concepts	6th	2010	English	1	1
2	9780133970777	Fundamentals of Database Systems	7th	2015	English	2	1
3	9780132350884	Clean Code	1st	2008	English	3	2

LOAN

loan_id	copy_id	member_id	staff_id	loan_date	due_date	return_date
1	1	1	1	2026-10-01	2026-10-15	2026-10-14
2	3	2	2	2026-10-05	2026-11-04	NULL
3	2	1	1	2026-10-10	2026-10-24	2026-10-27

Loan 3 was returned 3 days late, so a FINE row (loan_id 3, type Overdue, amount PHP 15.00 at PHP 5.00 per day) would be created.